EX25 · NumPy para álgebra lineal

Referencia ampliada: entradas, salidas, ejemplos ejecutables, errores típicos y testing.

LANGUAGE / ID	UNIT	HANDOUT
ES · EX25	algebra / vectors and linear systems	Guía de referencia Python

1 · Mapa mental

En Python una lista sirve para almacenar valores; un **ndarray** de NumPy representa datos numéricos con dimensiones explícitas. Un vector 2D suele tener forma **(2,)** y una matriz con dos filas y dos columnas tiene forma **(2, 2)**. La forma del dato es tan importante como sus números.

```
import numpy as np
u = np.array([2.0, 3.0], dtype=float)
A = np.array([[3.0, 1.0], [1.0, 2.0]], dtype=float)

print(u.shape)     # (2,)
print(A.shape)     # (2, 2)
print(u.ndim)      # 1
print(A.ndim)      # 2
```

Concepto	Ejemplo	Interpretación
shape	A.shape → (2, 2)	Nº de filas y columnas
ndim	u.ndim → 1	Nº de ejes del array
dtype	u.dtype	Tipo numérico almacenado
axis	axis=0 / axis=1	Dirección de cálculo en un array

2 · Productos que no deben confundirse

```
A = np.array([[3., 1.], [1., 2.]])
u = np.array([2., 3.])
print(A @ u)  # [9., 8.] -> matrix-vector product
print(A * u)  # [[6., 3.], [2., 6.]] -> broadcasting
print(u * u)  # [4., 9.] -> element-wise product
print(np.dot(u, u))  # 13.0 -> scalar dot product
```

Regla práctica: A @ u aplica la transformación lineal representada por A. A * u multiplica cada columna con el elemento correspondiente de u; no resuelve un sistema de fuerzas. Usa @ cuando la operación física sea una combinación lineal.

3 · np.linalg.solve(A, b): resolver A @ u = b

Entrada: matriz cuadrada A y vector b con dimensiones compatibles. **Salida:** vector u que satisface el sistema, si A es no singular. **Cuándo usarla:** cuando hay varias ecuaciones lineales con tantas incógnitas como ecuaciones.

```
A = np.array([[3., 1.], [1., 2.]])
b = np.array([9., 8.])
u = np.linalg.solve(A, b)
print(u)       # [2., 3.]
print(A @ u)   # [9., 8.]
```

> No uses **np.linalg.inv(A) @ b** como método habitual para resolver sistemas. **np.linalg.solve** evita formar explícitamente la inversa y suele ser más estable y eficiente.

4 · Normas y residuales

np.linalg.norm(x) devuelve, para un vector y por defecto, la norma euclídea (longitud). El residual indica cuánto se diferencia la fuerza reconstruida de la fuerza objetivo.

$$\mathbf{r} = \mathbf{A}\mathbf{u} - \mathbf{b}$$

$$\text{residual}_{abs} = \|\mathbf{r}\|_2$$

```
u = np.array([2., 3.])
b = np.array([9., 8.])
residual_vector = A @ u - b
absolute_error = float(np.linalg.norm(residual_vector))
relative_error = absolute_error / float(np.linalg.norm(b))
print(absolute_error)  # 0.0 (or nearly zero)
```

Importante: si ||b|| = 0, la expresión relativa anterior no está definida. En este ejercicio se requiere lanzar **ValueError**. No añadas silenciosamente un número pequeño al denominador.

5 · np.linalg.cond: sensibilidad

Entrada: matriz A y, opcionalmente, orden de norma **p=2**. **Salida:** escalar adimensional que estima cuánto pueden amplificarse pequeñas variaciones relativas en los datos al resolver el sistema.

```
print(np.linalg.cond(np.eye(2), 2))  # 1.0
B = np.array([[1., 1.], [1., 1.000001]])
print(np.linalg.cond(B, 2))       # approximately 4e6
```

Un número de condición grande alerta de sensibilidad. No es el residual: una solución puede reconstruir b casi exactamente y aun así cambiar mucho si b cambia muy poco.

6 · Validación de entradas y errores

Las funciones de ingeniería deben rechazar datos incompatibles. Utiliza **np.asarray(..., dtype=float)** para aceptar listas y convertirlas en arrays; revisa formas con **.shape** y valores no finitos con **np.isfinite**.

```
def check_inputs(A, b):
    A = np.asarray(A, dtype=float)
    b = np.asarray(b, dtype=float)
    if A.shape != (2, 2) or b.shape != (2,):
        raise ValueError("Expected A (2,2) and b (2,)")
    if not np.isfinite(A).all() or not np.isfinite(b).all():
        raise ValueError("All values must be finite")
    return A, b
```

La función anterior es un **ejemplo de referencia** que puedes adaptar; no resuelve por sí sola el ejercicio. Cuando **np.linalg.solve** encuentra una matriz singular, lanza **np.linalg.LinAlgError**.

7 · Comparar floats correctamente

Los floats almacenan aproximaciones binarias; por eso **0.1 + 0.2 == 0.3** puede ser False. Para cálculos numéricos usa tolerancias: **np.isclose** para escalares y **np.allclose** para arrays.

```
np.isclose(0.1 + 0.2, 0.3)   # True
np.allclose([2., 3.], [2.00000000001, 3.])  # True
np.allclose(A @ u, b, atol=1e-10, rtol=0)   # check tolerance
```

En **allclose**, los parámetros **atol** y **rtol** representan tolerancias absoluta y relativa. Fijar **rtol=0** permite exigir solo tolerancia absoluta en unidades físicas.

8 · Diccionarios con resultados y tipos

Un diccionario permite nombrar explícitamente magnitudes distintas. En el estudio de sensibilidad, incluye arrays para vectores y números de Python para cantidades escalares; puedes aplicar **float(...)** a los resultados de norma.

```
report = {
    "u_original": np.array([2., 3.]),
    "u_perturbed": np.array([2.1, 2.9]),
    "delta_u": np.array([0.1, -0.1]),
    "delta_u_norm": float(np.linalg.norm([0.1, -0.1])),
    # ... remaining fields ...
}
```

9 · pytest y depuración gradual

```
python -m pytest -q
python -m pytest -q -x
python -m pytest tests/test_algebra.py -vv
python main.py
```

Usa **-x** para detenerte en el primer fallo y **-vv** para mostrar el nombre completo de cada test. Distingue fallos de NotImplementedError (función pendiente), ValueError (entrada inválida) y LinAlgError (sistema singular).

10 · Resumen rápido

Necesitas…	Usa…
Crear vector o matriz	np.asarray(..., dtype=float) / np.array(...)
Comprobar tamaños	array.shape
Multiplicar matriz × vector	A @ u
Resolver ecuaciones lineales	np.linalg.solve(A, b)
Norma euclídea	np.linalg.norm(x)
Número de condición	np.linalg.cond(A, 2)
Comparar resultados numéricos	np.allclose / np.isclose
Evitar NaN y ±Inf	np.isfinite(...).all()